

NT120: Suggested Course Project Topics

Introduction to AI for Computer Networks and Cybersecurity

Class: NT120.R12

Instructors: Huynh Van Dang, Nguyen Khanh Thuat

This list suggests project topics for the course capstone project, each paired with a public dataset to help a group get started quickly. A group may also propose its own topic and dataset, subject to instructor approval, and may substitute an equivalent dataset for the one listed if it fits the topic better. Each entry names one dataset that is enough to start the project; a group is free to combine datasets or add a second one for comparison. Use the project notebook template distributed separately for the report structure and submission format.

Dataset links were checked in September 2026. If a link has moved, search for the dataset name together with the hosting organization named next to it; all datasets below are hosted by their original research group or a well known public repository.

1. Network Intrusion Detection from Flow-Based Features

Suggested dataset: CIC-IDS2017 or CSE-CIC-IDS2018, Canadian Institute for Cybersecurity (UNB):

<https://www.unb.ca/cic/datasets/ids-2017.html> and <https://www.unb.ca/cic/datasets/ids-2018.html>

Train a supervised classifier that separates benign traffic from common attacks such as denial of service, brute force, infiltration and web attacks, using flow-level features like duration, byte counts and packet timing. Compare at least two algorithms and report precision, recall and F1-score for each attack category, not only overall accuracy.

2. Anomaly-Based Intrusion Detection with Unsupervised Learning

Suggested dataset: NSL-KDD or UNSW-NB15: <https://www.unb.ca/cic/datasets/nsl.html> and

<https://research.unsw.edu.au/projects/unswnb15-dataset>

Train an unsupervised model, such as Isolation Forest, One-Class SVM or an autoencoder, on mostly normal traffic, then test how well it flags attack types it never saw labeled during training. Examine the trade-off between detection rate and false alarm rate as the anomaly threshold changes.

3. IoT Botnet Traffic Detection

Suggested dataset: N-BaIoT (UCI Machine Learning Repository) and Bot-IoT (UNSW Canberra):

<https://archive.ics.uci.edu/dataset/442/detection+of+iot+botnet+attacks+n+baiot> and

<https://research.unsw.edu.au/projects/bot-iot-dataset>

Detect Mirai- and Gafgyt-style botnet activity from network statistics captured on commercial IoT devices such as cameras and doorbells. Train one model per device and one shared model across devices, then compare whether a single global model generalizes as well as the device-specific ones.

4. DDoS Attack Detection and Classification

Suggested dataset: CIC-DDoS2019: <https://www.unb.ca/cic/datasets/ddos-2019.html>

Classify network flows into benign traffic and several reflection- and exploitation-based DDoS attack types, for example NTP, DNS and SYN flood. Identify a small set of features that keeps detection accurate while staying cheap enough to compute close to real time.

5. Encrypted and Darknet Traffic Classification

Suggested dataset: ISCX VPN-nonVPN and CIC-Darknet2020: <https://www.unb.ca/cic/datasets/vpn.html> and <https://www.unb.ca/cic/datasets/darknet2020.html>

Identify the application category of encrypted flows, for example browsing, streaming, VoIP or file transfer, using only packet size, timing and direction features, since the payload itself is unreadable. Extend the idea to separate darknet traffic (VPN and Tor) from regular traffic, and discuss the privacy considerations that come with darknet-labeled captures.

6. Static Malware Detection from Executable Features

Suggested dataset: EMBER, Elastic Malware Benchmark: <https://github.com/elastic/ember>

Train a classifier on features extracted from Windows PE file headers, imports and byte histograms to separate malicious from benign executables without running any file. Analyze which feature group, header fields, imported functions or byte statistics, contributes most to correct detection.

7. Memory-Based Malware Detection

Suggested dataset: CIC-MalMem2022: <https://www.unb.ca/cic/datasets/malmem-2022.html>

Detect the presence of malware families, including ransomware, spyware and trojans, from features extracted out of memory dumps rather than from the executable file itself. Report detection performance separately for each malware category, since some families leave a much weaker memory footprint than others.

8. Android Malware Detection

Suggested dataset: CICMalDroid2020: <https://www.unb.ca/cic/datasets/maldroid-2020.html>

Classify Android applications into benign and malware categories such as adware, banking trojan, SMS malware and riskware, using static permissions and API calls or dynamic runtime behavior. Identify which malware category is hardest to separate from benign apps and explain why.

9. Phishing Website Detection

Suggested dataset: UCI Phishing Websites dataset, with optional live validation from PhishTank: <https://archive.ics.uci.edu/dataset/327/phishing+websites> and <https://phishtank.org>

Train a classifier that flags phishing sites from URL and page-structure features, for example IP-based URLs, unusual subdomains, domain age and HTTPS usage. As a stretch goal, pull a handful of currently reported URLs from PhishTank and check whether the trained model still flags them correctly.

10. Botnet Command-and-Control Detection

Suggested dataset: CTU-13, Stratosphere Laboratory: <https://www.stratosphereips.org/datasets-ctu13>

Detect botnet-infected hosts using thirteen real network captures that mix labeled botnet, normal and background traffic. Consider representing hosts and their connections as a graph and combining graph-based features with standard flow statistics.

11. Federated Learning for Distributed Intrusion Detection

Suggested dataset: TON_IoT, or CIC-IDS2017 split across simulated clients: <https://research.unsw.edu.au/projects/toniot-datasets> and <https://www.unb.ca/cic/datasets/ids-2017.html>

Simulate several sites that train a shared intrusion detection model without sharing raw traffic, using a federated averaging setup. Compare detection accuracy and communication cost against a single model trained on centralized data.

12. Adversarial Robustness of Intrusion Detection Models

Suggested dataset: CIC-IDS2017: <https://www.unb.ca/cic/datasets/ids-2017.html>

Train a baseline intrusion detector, then craft small perturbations to attack-flow features designed to make the traffic look benign to the model. Measure how much detection accuracy drops under attack and evaluate at least one mitigation, such as adversarial training.

13. Explainable AI for Intrusion Detection

Suggested dataset: UNSW-NB15: <https://research.unsw.edu.au/projects/unsw-nb15-dataset>

Train an intrusion detection model and apply an explainability method such as SHAP or LIME to show which features drive individual predictions. Check whether the explanations line up with features a network analyst would expect for each attack type.

14. System Log Anomaly Detection

Suggested dataset: HDFS and BGL logs, Loghub collection: <https://github.com/logpai/loghub>

Parse raw system logs into structured event sequences, then train a model to flag sequences that look anomalous. Compare a simple frequency-based baseline against a model that accounts for event order.

15. Insider Threat Detection from User Activity Logs

Suggested dataset: CERT Insider Threat Test Dataset, Carnegie Mellon University SEI: https://kithub.cmu.edu/articles/dataset/Insider_Treat_Test_Dataset/12841247

Detect a small number of malicious insiders hidden among mostly normal login, file access, email and device activity. Discuss how the extreme class imbalance in this dataset shapes both the choice of model and the choice of evaluation metric.

Network Management, IoT, and Cloud Computing

16. Network Traffic Forecasting for Capacity Planning

Suggested dataset: GEANT and Abilene traffic matrices and topology traces, University of Surrey Open Research: <https://openresearch.surrey.ac.uk/esploro/outputs/dataset/GEANTAbilene-Network-Topology-Data-and-Traffic/99514653002346>

Forecast near-future link or router level traffic volume from historical traffic matrices, then turn the forecast into a simple capacity planning recommendation, for example which links come closest to saturation. Compare a classical time series baseline, such as ARIMA, against a learned model.

17. Intrusion Detection in Software-Defined Networks

Suggested dataset: InSDN: <https://www.kaggle.com/datasets/muhammadumarjavid/insdn-dataset-2020>

Detect attacks captured from an SDN testbed, including denial of service, probing and traffic aimed at the OpenFlow controller, and discuss how detection at the controller differs from detection on a traditional router. Compare against a topic 1 style detector trained on non-SDN flow data if time allows.

18. IoT Device Identification from Network Traffic

Suggested dataset: UNSW IoT Traffic Traces: <https://iotanalytics.unsw.edu.au/iottraces.html>

Identify the type of IoT device connected to a network, for example camera, speaker or sensor, purely from traffic statistics such as packet size, protocol and connection frequency, without reading payload content. Discuss how this kind of model could support automatic device inventory on a managed network.

19. Anomaly Detection in IoT Sensor Streams

Suggested dataset: Intel Berkeley Research Lab Sensor Data: <https://db.csail.mit.edu/labdata/labdata.html>

Train a model on readings from a real deployment of 54 wireless sensors, covering temperature, humidity, light and voltage, to flag sensor failures, unusual readings or nodes that stop reporting. Discuss how an anomaly here could mean either a real physical event or a faulty sensor, and how a deployed system would need to tell the two apart.

20. Cloud Workload Prediction for Auto-Scaling

Suggested dataset: Google Cluster Data, 2011 or 2019 trace: <https://github.com/google/cluster-data>

Predict short-term CPU or memory demand from historical cluster workload traces, then use the prediction to simulate a simple auto-scaling policy. Compare the number of under-provisioned and over-provisioned intervals against a policy that reacts only to the current load.

21. Anomaly Detection and Root Cause Localization in Cloud Microservices

*Suggested dataset: Alibaba Cluster Trace Microservices:
<https://github.com/alibaba/clusterdata/tree/master/cluster-trace-microservices-v2022>*

Use call graph and latency data from a real microservice deployment to detect performance anomalies and narrow down which service in the call chain is the likely cause. Discuss why this problem does not reduce to a single classifier, since a fault often surfaces first in a caller rather than in the service that is actually misbehaving.

A group choosing a topic outside this list should still follow the same expectations: a clearly stated problem, a dataset that a reader can access or reconstruct, at least one concrete algorithm, and results that are evaluated with metrics suited to the problem.